

Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №6

ШАБЛОНИ «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator»

Виконав
студент групи ІА–22:
Артамонов Дмитро Вікторович
Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Короткі теоретичні відомості 3
Реалізувати не менше 3-х класів для обраної теми 5
Реалізація шаблону за обраною темою 6
Діаграма реалізованих класів шаблону Декоратор 9
Результат виконання патерну 10
Проблему, яку допоміг вирішити шаблон Декоратор 10
Переваги застосування патерну Декоратор 11
Висновок 12

Тема: Шаблони «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator»

Мета: ознайомитися з шаблонами проектування «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator» та набути практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Хід роботи

..15 E-mail клієнт (singleton, builder, decorator, template method, interpreter, SOA)

Поштовий клієнт повинен нагадувати функціонал поштових програм Mozilla Thunderbird, The Bat і т.д. Він повинен сприймати і коректно обробляти

---

pop3/smtp/imap протоколи, мати функції автонастройки основних поштових провайдерів для України (gmail, ukr.net, i.ua), розділяти повідомлення на папки/категорії/важливість, зберігати чернетки незавершених повідомлень, прикріплювати і обробляти прикріплені файли.

Короткі теоретичні відомості

Принципи проектування SOLID

SOLID — набір принципів розробки об'єктно-орієнтованих систем, що сприяють створенню гнучкого, масштабованого і легкосупроводжуваного коду:

1. **Single Responsibility Principle (SRP)**: Клас повинен мати лише одну причину для зміни.
2. **Open/Closed Principle (OCP)**: Програмні сутності повинні бути відкриті для розширення, але закриті для модифікації.
3. **Liskov Substitution Principle (LSP)**: Об'єкти похідного класу повинні бути взаємозамінними з об'єктами базового класу.

4. **Interface Segregation Principle (ISP)**: Інтерфейси повинні бути специфічними і не примушувати реалізовувати непотрібні методи.
5. **Dependency Inversion Principle (DIP)**: Модулі високого рівня не повинні залежати від модулів низького рівня. Обидва повинні залежати від абстракцій.

Опис шаблонів

1. **Abstract Factory**: Використовується для створення груп взаємопов’язаних об'єктів без зазначення їх конкретних класів. Корисний у разі, коли система має змінювану конфігурацію.
2. **Factory Method**: Дозволяє створювати об'єкти, делегуючи їх ініціалізацію дочірнім класам. Це знижує залежність від конкретних класів у коді.
3. **Memento**: Застосовується для збереження і відновлення стану об'єкта без порушення принципу інкапсуляції. Це корисно для реалізації операцій "скасувати/повторити".
4. **Observer**: Забезпечує автоматичне сповіщення зацікавлених об'єктів (спостерігачів) про зміни в стані іншого об'єкта.
5. **Decorator**: Дозволяє динамічно додавати обов’язки об'єктам, створюючи ланцюжок вкладених об'єктів-декораторів. Це забезпечує більшу гнучкість порівняно зі спадкуванням.

Реалізувати не менше 3-х класів для обраної теми

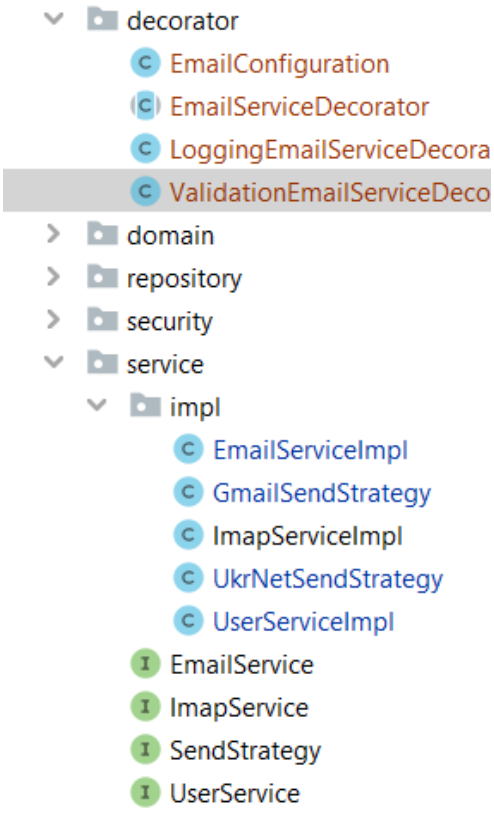


Рис. 1 Структура класів

Опис реалізованих класів шаблону декоратор:

1. Інтерфейс EmailService у контексті шаблону Декоратор виступає базовим контрактом, який реалізується як початковими сервісами, так і декораторами, дозволяючи обгортати базову функціональність додатковими поведінками без зміни основного коду.
2. Клас EmailServiceImpl виступає базовою реалізацією інтерфейсу EmailService, яку можуть обгортати декоратори для додавання нових функціональностей, таких як логування чи валідація, без зміни цього базового коду.
3. Клас `EmailServiceDecorator` є абстрактним базовим декоратором, який забезпечує обгортання реалізації `EmailService`, делегуючи всі виклики до оригінального сервісу, при цьому даючи можливість розширювати його функціональність у підкласах.
4. Клас `LoggingEmailServiceDecorator` є конкретним декоратором, який додає функціональність логування до базового сервісу `EmailService`, зберігаючи при цьому його основну поведінку.

5. Клас `ValidationEmailServiceDecorator` є конкретним декоратором, який додає перевірку валідності даних (як-от непусті адреси відправника і отримувача) до функціональності базового сервісу `EmailService`.
6. Клас `EmailConfiguration` є конфігураційним класом, який збирає декоратори `ValidationEmailServiceDecorator` і `LoggingEmailServiceDecorator` навколо базового сервісу `EmailService`, створюючи стек декорованих об'єктів із розширеною функціональністю.

Реалізація шаблону за обраною темою

```
import com.example.emailapp.domain.Email;
import com.example.emailapp.domain.MailBox;
import com.example.emailapp.web.dto.email.DetailedReceivedEmail;
import java.io.UnsupportedEncodingException;
import javax.mail.MessagingException;

23 usages  4 implementations  Dima Artamonov
public interface EmailService {

    4 usages  4 implementations  Dima Artamonov
    Email sendEmail(Email email) throws MessagingException, UnsupportedEncodingException;

    2 usages  2 implementations  Dima Artamonov
    DetailedReceivedEmail getSpecificEmail(String account, String folderName, int msgnum)
            throws Exception;

    2 usages  2 implementations  Dima Artamonov
    boolean addEmailConfiguration(MailBox mailBox);
}
```

Рис 2. Інтерфейс EmailService

1. Клас EmailService є інтерфейсом, який визначає контракт для сервісу роботи з електронною поштою. Він містить три основні методи:

    - **sendEmail** - відповідає за відправку електронних листів і може викидати виключення, пов’язані з поштовими протоколами чи кодуванням.
    - **getSpecificEmail** - забезпечує отримання детальної інформації про конкретний електронний лист із вказаної папки для заданого акаунту.
    - **addEmailConfiguration** - дозволяє додати налаштування для поштової скриньки.

Цей інтерфейс задає структуру для реалізації функціональності роботи з електронною поштою, яку можна розширювати або декорувати для додаткових можливостей.

```
2 usages  2 inheritors
public abstract class EmailServiceDecorator implements EmailService {
    4 usages
    protected final EmailService emailService;
    2 usages
    public EmailServiceDecorator(EmailService emailService) { this.emailService = emailService; }
    4 usages  2 overrides
    @Override
    public Email sendEmail(Email email) throws MessagingException, UnsupportedEncodingException {
        return emailService.sendEmail(email);
    }
    2 usages
    @Override
    public DetailedReceivedEmail getSpecificEmail(String account, String folderName, int msgnum)
            throws Exception {
        return emailService.getSpecificEmail(account, folderName, msgnum);
    }
    2 usages
    @Override
    public boolean addEmailConfiguration(MailBox mailBox) {
        return emailService.addEmailConfiguration(mailBox);
    }
}
```

Рис. 3 абстрактний клас EmailServiceDecorator

2. Клас EmailServiceDecorator є абстрактним класом, що реалізує інтерфейс EmailService та використовує шаблон проектування Decorator. Він делегує виклики методів до об’єкта EmailService, дозволяючи розширювати або модифікувати його функціональність без зміни вихідного коду.

    - **sendEmail(Email email)**: Делегує відправку електронного листа до базової реалізації.
    - **getSpecificEmail(String account, String folderName, int msgnum)**: Повертає конкретний електронний лист, делегуючи виклик до базової реалізації.
    - **addEmailConfiguration(MailBox mailBox)**: Додає конфігурацію поштової скриньки через базову реалізацію.

```java
public class LoggingEmailServiceDecorator extends EmailServiceDecorator {

    2 usages
    private static final Logger logger = LoggerFactory.getLogger(LoggingEmailServiceDecorator.class);

    1 usage
    public LoggingEmailServiceDecorator(EmailService emailService) {
        super(emailService);
    }

    4 usages
    @Override
    public Email sendEmail(Email email) throws MessagingException, UnsupportedEncodingException {
        logger.info("Sending email: {}", email);
        Email sentEmail = super.sendEmail(email);
        logger.info("Email sent successfully: {}", sentEmail);
        return sentEmail;
    }
}
```

Рис. 4 клас LoggingEmailServiceDecorator

3. Клас LoggingEmailServiceDecorator є реалізацією шаблону Decorator для EmailService, яка додає логування процесу відправки електронних листів. Перед та після виклику методу sendEmail логуються відповідні повідомлення з деталями листа.
    - **sendEmail(Email email)**: Додає логування перед та після відправлення електронного листа, делегуючи основну функціональність базовому `EmailService`.

```java
2 usages
public class ValidationEmailServiceDecorator extends EmailServiceDecorator {

    1 usage
    public ValidationEmailServiceDecorator(EmailService emailService) {
        super(emailService);
    }

    4 usages
    @Override
    public Email sendEmail(Email email) throws MessagingException, UnsupportedEncodingException {
        if (StringUtils.isBlank(email.getSenderEmail()) || StringUtils.isBlank(email.getRecipientEmail())) {
            System.out.println("Sender and recipient emails must not be blank");
            throw new IllegalArgumentException("Sender and recipient emails must not be blank");
        }
        return super.sendEmail(email);
    }
}
```

Рис. 5 клас ValidationEmailServiceDecorator

4. Клас ValidationEmailServiceDecorator є реалізацією шаблону Decorator для EmailService, яка додає перевірку валідності електронних листів перед їх відправленням. Він перевіряє, чи заповнені адреси відправника та

отримувача, і в разі відсутності даних генерує виняток. Якщо перевірка успішна, виклик методу sendEmail делегується базовому EmailService.

Діаграма реалізованих класів шаблону Декоратор

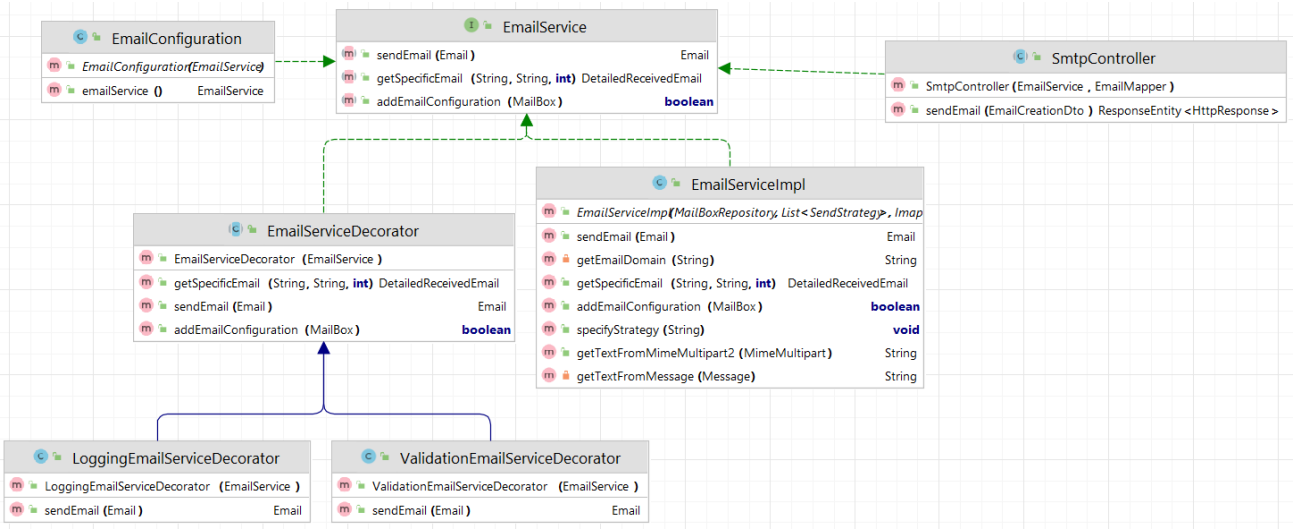


Рис. 6 Діаграма класів

Опис основних компонентів:

1. Інтерфейс EmailService забезпечує функціонал для надсилання, отримання електронних листів та управління конфігураціями поштових скриньок;
2. Клас EmailServiceImpl реалізує інтерфейс EmailService, забезпечуючи функціонал для надсилання електронних листів із використанням стратегій, отримання деталей конкретного листа через IMAP, а також налаштування поштових скриньок на основі їх домену;
3. EmailServiceDecorator є абстрактним декоратором для інтерфейсу EmailService, який надає можливість розширювати функціональність базового сервісу електронної пошти, делегуючи виклики методів до об'єкта EmailService, що передається у його конструктор.

Опис декораторів:

1. Клас `LoggingEmailServiceDecorator` є реалізацією декоратора для `EmailService`, який додає логування процесу надсилання електронних листів, записуючи інформацію до та після виконання операції `sendEmail`.
2. Клас ValidationEmailServiceDecorator є реалізацією декоратора для `EmailService`, який додає перевірку валідності даних електронного листа перед його відправленням, зокрема перевіряє, щоб адреси відправника та отримувача не були порожніми.

Опис класів, які використовують шаблон декоратор:

1. Клас EmailConfiguration є конфігураційним класом, який забезпечує створення декорованого сервісу електронної пошти. У ньому через метод emailService() об'єднуються декоратори:
   - **ValidationEmailServiceDecorator** – додає перевірку валідності електронних листів.
   - **LoggingEmailServiceDecorator** – додає логування процесу відправлення.

   Результат повертається як повністю декорований сервіс під іменем decorator-service.
2. Клас SmtpController є REST-контролером, який обробляє HTTP-запити для надсилання електронних листів через ендпоінт /api/v1/send. Він приймає дані про лист у вигляді DTO (EmailCreationDto), використовує сервіс електронної пошти (decorator-service) для відправлення та повертає відповідь у вигляді об'єкта HttpResponse.

Результат виконання патерну

```
- [nio-9090-exec-1] c.e.e.d.LoggingEmailServiceDecorator     : Sending email: com.examp
- [nio-9090-exec-1] c.e.e.d.LoggingEmailServiceDecorator     : Email sent successfully:
```

Рис. 7 результат виконання

```
2024-12-14T00:28:11.756+02:00  INFO 24340 --- [nio-9090-exec-3] c.e.e.d.LoggingEmailServiceDecorator     : Sending email:
Sender and recipient emails must not be blank
2024-12-14T00:28:11.761+02:00 ERROR 24340 --- [nio-9090-exec-3] o.a.c.c.C.[.[.[/].[dispatcherServlet]    : Servlet.servic

java.lang.IllegalArgumentException Create breakpoint : Sender and recipient emails must not be blank
```

Рис.8 результат виконання

Проблему, яку допоміг вирішити шаблон Декоратор

Шаблон Декоратор допоміг вирішити проблему розширення функціональності сервісу електронної пошти без модифікації базового коду. У цьому проєкті він дозволив:

1. Додати перевірку даних (ValidationEmailServiceDecorator) перед відправленням листа, забезпечуючи цілісність вхідних даних.
2. Додати логування (LoggingEmailServiceDecorator) процесу надсилання, що спрощує моніторинг і налагодження.

При цьому базовий сервіс EmailService залишився незмінним, що сприяє підтримці принципів SOLID, зокрема відкритості для розширення та закритості для змін (OCP). Декоратори забезпечили гнучкість у додаванні нової поведінки шляхом композиції, а не успадкування.

Переваги застосування патерну Декоратор

Декоратор має кілька ключових переваг, які роблять його дуже корисним для розширення функціональності об'єктів без зміни їх базового коду:

1. Гнучкість: Декоратор дозволяє додавати нову функціональність об'єктам динамічно, змінюючи їх поведінку без зміни вихідного коду. Це особливо корисно при необхідності розширення функціональності без створення великої кількості підкласів.

2. Розширення без змін в існуючому коді: Ви можете додавати нові можливості, не змінюючи вже існуючі класи або методи. Це забезпечує більшу стабільність і знижує ймовірність помилок, оскільки мінімізується модифікація коду.

3. Композиція замість успадкування: Замість створення великої кількості підкласів для різних варіантів поведінки, декоратори дозволяють комбінувати різні функціональні можливості в об'єктах. Це знижує складність і кількість класів.

4. Легкість у тестуванні: Кожен декоратор можна тестувати окремо, що дозволяє ізолювати логіку і забезпечити кращу підтримку тестування. Це особливо корисно в великих проектах, де перевірка всіх варіантів поведінки через одиничні класи може бути складною.

5. Читабельність коду: Кожен декоратор реалізує одну чітку функціональність, що дозволяє підтримувати код чистим і легким для розуміння. Це зменшує складність і покращує підтримку проекту в довгостроковій перспективі.

6. Реалізація множинних варіантів поведінки: Завдяки використанню декораторів, можна створювати різні варіанти поведінки для об'єкта, комбінуючи кілька декораторів. Це дозволяє легко налаштовувати функціональність в залежності від потреби, зберігаючи при цьому основний клас простим і універсальним.

7. Підтримка принципу відкритості/закритості (OCP): Патерн декоратор підтримує один із принципів SOLID, який говорить, що класи повинні

бути відкриті для розширення, але закриті для змін. Це дозволяє безпечно додавати нові функціональності без потреби змінювати вже існуючий код.

Посилання на репозиторій: https://github.com/qjsoq/Email-API

Висновок

У ході виконання лабораторної роботи був реалізований патерн декоратор який допоміг ефективно розширити функціональність сервісу електронної пошти, додавши перевірку валідності та логування без зміни базового коду. Це дозволило зберегти гнучкість і масштабованість проекту, забезпечуючи простоту в тестуванні та підтримці розширеної поведінки.